

Pontifícia Universidade Católica de Minas Gerais

Instituto de Ciências Exatas e Informática (ICEI)
Engenharia de Software — Redes de Computadores
Professor: Diego Rocha

TRABALHO PRÁTICO

Jogo da Velha via Sockets TCP em Java

Integrante	GitHub
Henrique Carvalho	@henriquegdc
Renato Douglas	@RenatoDNS
Vicenzo Fonseca	@vicenzofms

Repositório:	https://github.com/vicenzofms/TicTacToe-Sockets
Linguagem:	Java (JDK 8+)
Protocolo:	TCP / Sockets
Data:	Maior de 2026

1. Introdução

Este trabalho prático tem como objetivo aplicar o conceito de sockets em redes de computadores, implementando uma comunicação entre processos situados em sistemas distintos por meio do mecanismo de socket TCP. A aplicação escolhida é o clássico Jogo da Velha (Tic-Tac-Toe) para dois jogadores em rede, desenvolvida inteiramente em Java.

A arquitetura adotada é cliente-servidor centralizada: um processo servidor aguarda conexões na porta 5000 (TCP) e, a cada dois clientes conectados, cria uma sessão de jogo dedicada. O servidor atua como árbitro da partida — recebe as jogadas enviadas pelos clientes, valida-as, atualiza o estado do tabuleiro e retransmite o novo estado e os comandos de status para ambos os jogadores. Os clientes limitam-se a exibir o estado do tabuleiro no console e a ler a entrada do usuário.

A comunicação é feita com um protocolo de texto puro, um comando por linha, no formato TIPO:VALOR, o que facilita a depuração com ferramentas como telnet ou netcat. O servidor suporta múltiplas partidas simultâneas graças ao uso de threads por jogador.

Funcionalidades Implementadas

- **Conexão TCP ponto-a-ponto:** Dois jogadores conectam-se ao servidor via socket TCP; o servidor faz o pareamento automático.
- **Protocolo textual customizado:** Comandos como MOVE, BOARD, STATUS, ASSIGN, ERROR, QUIT e REMATCH trafegam como texto simples, um por linha.
- **Validação de jogadas no servidor:** O servidor verifica turno, posição válida (1-9) e posição livre; erros são reportados apenas ao jogador infrator.

- **Detecção de vitória, empate e desconexão:** O servidor identifica as oito linhas vencedoras, tabuleiro cheio (empate) e desconexão inesperada.
- **Revanche (rematch):** Ao fim de cada partida, o servidor pergunta aos dois jogadores se desejam nova partida; o jogo só reinicia se ambos aceitarem.
- **Interface colorida no console:** O cliente usa códigos de escape ANSI para colorir peças (X em vermelho, O em azul) e mensagens de status.
- **Containerização com Docker:** O servidor pode ser executado em container Docker sem dependências adicionais no host.

2. Implementação

2.1 Estrutura do Projeto

O projeto é organizado em três pacotes Java, cada um com responsabilidade clara:

Pacote / Arquivo	Responsabilidade
common/Protocol.java	Constantes e helper para montar os comandos do protocolo de comunicação.
server/Server.java	Ponto de entrada do servidor. Aceita conexões TCP e cria GameSessions.
server/Board.java	Representa e manipula o tabuleiro 3x3 (células, jogadas, vitória, empate).
server/GameSession.java	Gerencia uma partida completa: threads dos jogadores, turnos, protocolo e revanche.
client/Client.java	Processo do jogador: conecta ao servidor, exibe o tabuleiro e lê a entrada do usuário.

2.2 Arquitetura e Fluxo de Comunicação

A arquitetura é cliente-servidor centralizada com threads por jogador. O diagrama abaixo ilustra o fluxo de comunicação entre os processos durante uma partida:

JOGADOR X (Client)	SERVIDOR (GameSession)	JOGADOR O (Client)
conecta →	accept() / PlayerHandler X	
	accept() / PlayerHandler O	← conecta
ASSIGN:X ←	envia símbolos	← ASSIGN:O
BOARD:... ←	broadcast estado inicial	← BOARD:...
STATUS:YOUR_TURN ←	notifica turnos	← STATUS:WAIT
→ MOVE:5	valida, atualiza Board	
BOARD:... ←	broadcast novo estado	← BOARD:...
STATUS:WAIT ←	troca de turno	← STATUS:YOUR_TURN
...	...	...
STATUS:WIN ←	detecta vencedor	← STATUS:LOSE

Diagrama de sequência simplificado da comunicação entre os processos.

2.3 Protocolo de Comunicação

Toda comunicação ocorre via texto puro (UTF-8), uma mensagem por linha, no formato TIPO:VALOR. Isso torna o protocolo simples, legível e facilmente depurável com ferramentas como netcat ou telnet.

Direção	Comando	Descrição
C → S	MOVE:5	Jogador faz uma jogada na posição 5 (1 a 9).
C → S	REMATCH:sim	Jogador aceita a revanche.
C → S	QUIT	Jogador encerra a conexão voluntariamente.
S → C	ASSIGN:X	Servidor informa o símbolo atribuído ao jogador.
S → C	BOARD:XO_...	Estado serializado do tabuleiro (9 chars, '_' = vazio).
S → C	STATUS:YOUR_TURN	É a vez deste jogador jogar.
S → C	STATUS:WAIT	Aguardando jogada do oponente.
S → C	STATUS:WIN/LOSE/DRAW	Resultado final da partida.
S → C	STATUS:REMATCH_REQUEST	Servidor solicita resposta sobre revanche.
S → C	STATUS:OPONENT_LEFT	O oponente desconectou da partida.
S → C	MESSAGE:...	Mensagem informativa ao jogador.
S → C	ERROR:...	Erro na jogada ou comando inválido recebido.

Tabela completa dos comandos do protocolo de comunicação.

2.4 Estruturas de Dados

A estrutura central é o array `char[] cells` de 9 posições em `Board.java`, representando o tabuleiro 3x3. Cada posição armazena 'X', 'O' ou '_' (vazio). O mapeamento de índices para o tabuleiro é:

0	1	2
3	4	5
6	7	8

Índices do array `cells[]` mapeados no tabuleiro. O usuário digita posição 1-9 (índice + 1).

As variáveis de estado de `GameSession` são:

- **playerX / playerO:** Instâncias de `PlayerHandler` — thread interna com socket, streams de I/O e símbolo do jogador.
- **board:** Instância de `Board` com o estado atual do tabuleiro.
- **currentTurn:** Char 'X' ou 'O' indicando de quem é a vez.
- **finished:** Flag booleana que indica se a partida terminou.
- **closing:** Flag para evitar dupla execução do fechamento de sockets.
- **rematchX / rematchO:** Boolean nullable armazenando a resposta de cada jogador sobre a revanche (null = ainda não respondeu).

2.5 Código-Fonte Comentado

Protocol.java — Constantes e montagem de comandos

```
package common;

public final class Protocol {
    // Tipos de mensagens Cliente → Servidor
```

```

public static final String MOVE      = "MOVE";
public static final String REMATCH  = "REMATCH";
public static final String QUIT     = "QUIT";

// Tipos de mensagens Servidor → Cliente
public static final String BOARD    = "BOARD";
public static final String STATUS   = "STATUS";
public static final String MESSAGE  = "MESSAGE";
public static final String ASSIGN   = "ASSIGN";
public static final String ERROR    = "ERROR";

// Valores de STATUS
public static final String YOUR_TURN    = "YOUR_TURN";
public static final String WAIT         = "WAIT";
public static final String WIN          = "WIN";
public static final String DRAW         = "DRAW";
public static final String LOSE         = "LOSE";
public static final String OPPONENT_LEFT = "OPPONENT_LEFT";
public static final String REMATCH_REQUEST = "REMATCH_REQUEST";

public static final String SEPARATOR = ":";
private Protocol() { } // impede instanciação

// Monta "TIPO:VALOR"
public static String command(String type, String value) {
    return type + SEPARATOR + value;
}
}

```

Board.java — Tabuleiro e lógica de jogo

```

package server;

public class Board {
    private static final int SIZE = 9;
    private final char[] cells = new char[SIZE]; // '_' = vazio

    public Board() { reset(); }

    // Preenche todas as células com '_'
    public void reset() {
        for (int i = 0; i < SIZE; i++) cells[i] = '_';
    }

    // Faz a jogada; retorna false se posição inválida ou ocupada
    public boolean makeMove(int position, char symbol) {
        if (position < 0 || position >= SIZE || cells[position] != '_')
            return false;
        cells[position] = symbol;
        return true;
    }

    // Verifica as 8 linhas vencedoras para o símbolo dado
    public boolean hasWinner(char symbol) {
        int[][] lines = {
            {0,1,2}, {3,4,5}, {6,7,8}, // linhas
            {0,3,6}, {1,4,7}, {2,5,8}, // colunas
            {0,4,8}, {2,4,6}           // diagonais
        };
        for (int[] line : lines)
            if (cells[line[0]]==symbol && cells[line[1]]==symbol
                && cells[line[2]]==symbol)
                return true;
        return false;
    }

    // Empate: nenhuma célula vazia
    public boolean isFull() {
        for (char c : cells) if (c == '_') return false;
        return true;
    }

    // Serializa: "XO_X_O_X" (9 chars)

```

```

    public String serialize() { return new String(cells); }
}

```

Server.java — Aceita conexões e cria partidas

```

package server;

import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;

public class Server {
    private static final int DEFAULT_PORT = 5000;

    public static void main(String[] args) {
        int port = args.length > 0 ? Integer.parseInt(args[0]) : DEFAULT_PORT;

        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("Servidor iniciado na porta " + port + ".");

            while (true) {
                // Aguarda o 1º jogador (X) - bloqueia aqui
                Socket playerX = serverSocket.accept();
                System.out.println("Jogador X: " + playerX.getRemoteSocketAddress());

                // Aguarda o 2º jogador (O)
                Socket playerO = serverSocket.accept();
                System.out.println("Jogador O: " + playerO.getRemoteSocketAddress());

                // Cria sessão e inicia threads internas
                new GameSession(playerX, playerO).start();
                System.out.println("Nova partida iniciada.");
                // Volta ao topo: aguarda próximo par de jogadores
            }
        } catch (IOException e) {
            System.err.println("Erro no servidor: " + e.getMessage());
        }
    }
}

```

GameSession.java — Controle da partida (trechos principais)

```

// Inicia a partida: envia símbolos, mensagem e estado inicial
public void start() {
    playerX.send(Protocol.command(Protocol.ASSIGN, "X"));
    playerO.send(Protocol.command(Protocol.ASSIGN, "O"));
    broadcast(Protocol.command(Protocol.MESSAGE, "Partida iniciada. Jogador X começa."));
    sendState();
    new Thread(playerX, "player-x-handler").start();
    new Thread(playerO, "player-o-handler").start();
}

// Processa jogada (synchronized: evita race condition entre threads)
private synchronized void handleMove(PlayerHandler player, String value) {
    if (finished) return;

    if (player.symbol != currentTurn) {
        player.send(Protocol.command(Protocol.ERROR, "Espere sua vez!"));
        sendState(); return;
    }

    int position = Integer.parseInt(value.trim()) - 1; // converte 1-9 para 0-8

    if (!board.makeMove(position, player.symbol)) {
        player.send(Protocol.command(Protocol.ERROR, "Posição inválida ou ocupada."));
        sendState(); return;
    }

    if (board.hasWinner(player.symbol)) {
        finished = true; sendBoard();
        player.send(Protocol.command(Protocol.STATUS, Protocol.WIN));
        opponentOf(player).send(Protocol.command(Protocol.STATUS, Protocol.LOSE));
    }
}

```

```

        requestRematch(); return;
    }

    if (board.isFull()) {
        finished = true; sendBoard();
        broadcast(Protocol.command(Protocol.STATUS, Protocol.DRAW));
        requestRematch(); return;
    }

    currentTurn = (currentTurn == 'X') ? 'O' : 'X'; // alterna turno
    sendState();
}

// Envia tabuleiro + status de turno para ambos os jogadores
private void sendState() {
    sendBoard();
    playerX.send(Protocol.command(Protocol.STATUS,
        currentTurn == 'X' ? Protocol.YOUR_TURN : Protocol.WAIT));
    playerO.send(Protocol.command(Protocol.STATUS,
        currentTurn == 'O' ? Protocol.YOUR_TURN : Protocol.WAIT));
}

```

Client.java — Interface do jogador (trechos principais)

```

// Loop principal: lê mensagens do servidor e reage
String line;
while ((line = serverIn.readLine()) != null) {
    boolean shouldContinue = handleServerMessage(line, keyboard, out);
    if (!shouldContinue) break;
}

// Renderiza o tabuleiro com cores ANSI no console
private void printBoard(String board) {
    System.out.print("\033[H\033[2J"); // limpa a tela
    for (int row = 0; row < 3; row++) {
        int base = row * 3;
        System.out.println(" " + cell(board, base) + " | "
            + cell(board, base+1) + " | " + cell(board, base+2));
        if (row < 2) System.out.println("----+----+----");
    }
}

// Célula colorida: X=vermelho, O=azul, vazio=número cinza
private String cell(String board, int index) {
    if (board.charAt(index) == '_') return GREY + (index+1) + RESET;
    char c = board.charAt(index);
    return (c=='X' ? RED : BLUE) + c + RESET;
}

// Trata STATUS:YOUR_TURN — lê entrada e envia MOVE
case Protocol.YOUR_TURN:
    System.out.print("Sua vez. Escolha uma posicao (1-9): ");
    String input = keyboard.readLine();
    if (input == null || input.trim().equalsIgnoreCase("sair")) {
        out.println(Protocol.QUIT);
        return false;
    }
    out.println(Protocol.command(Protocol.MOVE, input.trim()));
    return true;

```

3. Listagem de Testes Executados

3.1 Ambiente de Testes

Os testes foram realizados em ambiente Linux (Ubuntu 22.04 via WSL2) com Java 21 e também utilizando o Dockerfile. Para os testes remotos, dois computadores na mesma rede local foram utilizados, sendo um executando o servidor e outro executando os dois clientes.

3.2 Compilação e Inicialização

```
# Compilação
$ javac -d out src/common/Protocol.java src/server/Board.java \
    src/server/GameSession.java src/server/Server.java \
    src/client/Client.java

# Resultado: diretório out/ criado sem erros
$ ls out/
common/  server/  client/

# Terminal 1 – Servidor
$ java -cp out server.Server
Servidor iniciado na porta 5000.
Aguardando jogadores...

# Terminal 2 – Jogador X
$ java -cp out client.Client localhost 5000
Conectado ao servidor localhost:5000.

# Terminal 3 – Jogador O
$ java -cp out client.Client localhost 5000
Conectado ao servidor localhost:5000.
```

Resultado: Servidor imprimiu confirmações de conexão e a partida iniciou automaticamente.

3.3 Vitória do Jogador X (linha superior 0-1-2)

Sequência de jogadas: X→1, O→4, X→2, O→5, X→3 (vitória na linha superior).

```
X | X | X
---+---+---
O | O | 6
---+---+---
7 | 8 | 9

# Terminal Jogador X:
Você venceu!
Deseja jogar novamente? (s/n):

# Terminal Jogador O:
Você perdeu.
Deseja jogar novamente? (s/n):
```

Resultado: Vitória detectada corretamente na linha 0-1-2 após 5 jogadas.

3.4 Empate

Sequência: X→5, O→1, X→2, O→8, X→7, O→3, X→6, O→4, X→9.

```
O | X | O
---+---+---
O | X | X
---+---+---
X | O | X

# Ambos recebem:
Empate!
Deseja jogar novamente? (s/n):
```

Resultado: Empate detectado corretamente ao preencher todas as 9 células sem vencedor.

3.5 Jogada Fora de Turno

```
# Jogador O tenta enviar MOVE:3 durante turno de X
# Resposta do servidor (apenas para O):
Erro: Espere sua vez!
# Tabuleiro reenviado sem alteração – estado preservado
```

Resultado: Servidor rejeitou a jogada e não alterou o estado da partida.

3.6 Posição Inválida / Ocupada

```
# Jogada em posição já ocupada
MOVE:5 → Erro: Posição invalida ou ocupada. Use uma posição de 1 a 9.

# Entrada não-numérica
```

```
MOVE:abc → Erro: Jogada invalida. Use um numero de 1 a 9.

# Posição fora do intervalo
MOVE:0 → Erro: Posição invalida ou ocupada.
MOVE:10 → Erro: Posição invalida ou ocupada.
```

Resultado: Todos os casos de entrada inválida tratados sem travar a sessão.

3.7 Desconexão Durante a Partida

```
# Jogador X fecha o terminal (Ctrl+C)
# Servidor detecta IOException no readLine()

# Jogador O recebe:
STATUS:OPPONENT_LEFT
O outro jogador desconectou. Partida encerrada.
# Conexão de O é fechada pelo servidor
```

Resultado: Desconexão detectada corretamente; oponente notificado e sessão encerrada.

3.8 Revanche

```
# Cenário A: ambos aceitam (digitam 's')
Voce aceitou a revanche.
Aguardando resposta do outro jogador...
Revanche iniciada. Jogador X começa.
# Tabuleiro reiniciado, jogo recomeça

# Cenário B: um jogador recusa (digita 'n')
Revanche recusada. Partida encerrada.
# Ambas as conexões são fechadas pelo servidor
```

Resultado: Lógica de revanche funcionou corretamente nos dois cenários.

3.9 Teste via Docker

```
# Build da imagem
$ docker build -t tictactoe-server .

# Servidor em container
$ docker run --rm -p 5000:5000 tictactoe-server
Servidor iniciado na porta 5000.

# Clientes no host (sem alteração)
$ java -cp out client.Client localhost 5000
```

Resultado: Servidor funcional em container Docker; clientes conectaram sem nenhuma modificação.

4. Conclusão

O trabalho foi concluído com sucesso, resultando em um jogo da velha multijogador funcional utilizando sockets TCP em Java.

A implementação demonstra na prática os principais conceitos de redes de computadores: estabelecimento de conexão TCP, comunicação bidirecional full-duplex, serialização de dados e gerenciamento de sessões concorrentes com threads.

A separação entre Protocol, Board, GameSession e Client manteve o código organizado e fácil de manter. O uso de synchronized nas operações críticas de GameSession evitou condições de corrida entre as threads dos jogadores.

Principais Dificuldades Encontradas

- **Sincronização de threads:** O acesso concorrente às variáveis de estado da GameSession (board, currentTurn, finished) exigiu uso cuidadoso de synchronized para evitar race conditions e comportamentos imprevisíveis.

- **Protocolo sem delimitadores complexos:** Optou-se por mensagens simples de texto (TIPO:VALOR por linha), o que simplificou o parsing mas exigiu atenção com o índice do separador ':' em valores que poderiam contê-lo.
- **Deteção de desconexão:** Java não lança exceção imediata ao fechar a conexão TCP; foi necessário tratar o retorno null do readLine() além de IOException para capturar todos os casos de desconexão.
- **Sincronização de revanche:** A lógica de revanche requer esperar a resposta dos dois jogadores antes de decidir; o uso de Boolean nullable (null = sem resposta) simplificou esta sincronização.
- **Codificação ANSI no Windows:** Os códigos de escape ANSI para cores no cliente funcionam em terminais Linux/macOS mas podem não funcionar no Windows CMD padrão sem configuração adicional (ANSI.SYS ou Windows Terminal).

Em suma, o projeto atingiu todos os requisitos do enunciado: comunicação entre processos em sistemas distintos via socket, protocolo definido, validação de regras no servidor, detecção de vitória/empate/desconexão e interface de console funcional.

5. Bibliografia

- [1] **Oracle Corporation.** Java SE Documentation — Class Socket, ServerSocket, Thread. Disponível em: <https://docs.oracle.com/en/java/docs/api/>. Acesso: maio 2026.
- [2] **Tanenbaum, A. S.; Wetherall, D. J.** Redes de Computadores. 5ª ed. São Paulo: Pearson Prentice Hall, 2011.
- [3] **Kurose, J. F.; Ross, K. W.** Redes de Computadores e a Internet — Uma Abordagem Top-Down. 7ª ed. São Paulo: Pearson, 2013.
- [4] **Baeldung.** Java Sockets — Guide. Disponível em: <https://www.baeldung.com/a-guide-to-java-sockets>. Acesso: maio 2026.
- [5] **Docker Inc.** Docker Documentation — Dockerfile reference. Disponível em: <https://docs.docker.com/reference/dockerfile/>. Acesso: maio 2026.
- [6] **GitHub.** Repositório do projeto — TicTacToe-Sockets. Disponível em: <https://github.com/vicenzofms/TicTacToe-Sockets>. Acesso: maio 2026.